

CampusHire — Prompt Engineering Playbook

All prompts from the *Prompt Engineering for Project Development* playbook, filled in specifically for CampusHire. Use each prompt at the appropriate phase. Do not run them all at once.

How to Use This File

Copy a prompt block, paste it into a new Kiro/AI conversation, and run it. Keep the AI's output as a project artifact. After each major phase, review the output yourself before moving forward.

Recommended Development Flow

Idea → Discovery → Requirements → Scope → Architecture → Documentation → UX/UI → Frontend MVP → Backend MVP → Integration → Testing → Security → Deployment → Observability → Iteration

Phase	Main goal	Primary output
0. Project context	Give AI the right context	Project brief / rules
1. Discovery	Understand problem & users	Problem statement, personas
2. Requirements	Define what must exist	PRD + acceptance criteria
3. MVP scope	Cut unnecessary work	MVP / V1 / later list
4. Architecture	Choose simple structure	Architecture + data flow
5. Documentation	Create source of truth	Docs / decisions / API plan
6. UX/UI	Design usable experience	Flows + screens + components
7. Frontend MVP	Build visible product	Working frontend
8. Backend MVP	Build business logic	API + DB + auth
9. Integration	Connect everything	End-to-end MVP
10. Quality	Find and fix failures	Tests + security checklist
11. Deployment	Ship safely	Production deployment
12. Iteration	Improve with evidence	Backlog + V2 roadmap

Phase 0 — Master Project Context Prompt

Use this at the start of any new AI conversation about CampusHire. It prevents the AI from making assumptions and sets how you want it to work.

You are my senior software architect, product manager, UX designer, full-stack engineer,
QA engineer, and technical documentation partner.

I am building: CampusHire

One-line idea: A campus placement management platform that connects students, department
admins, and super admins to streamline the entire placement process – from student
profile
building to drive management, application tracking, and placement reporting.

Target users:

- Students: Final-year and pre-final-year engineering/degree college students looking for placement opportunities
- Department Admins: Placement coordinators or department faculty responsible for managing their department's student data, drives, and forms
- Super Admin: The central placement cell or institution head who manages all departments, admin accounts, and institution-level settings

Problem:

College placement processes are managed through scattered spreadsheets, WhatsApp messages, and manual processes. Students don't know their placement readiness. Admins can't track student profiles efficiently. There is no single system for drive posting, form collection, application tracking, and reporting.

Main goal: Build a working full-stack platform where students can build their profiles and track placement drives, department admins can manage students and post drives, and super admins can manage the entire institution.

Preferred stack: React + Vite (frontend), FastAPI + Python (backend), PostgreSQL (database), JWT authentication with OTP email verification, local filesystem for file storage (S3-ready).

Constraints:

- Solo developer / small team
- Local development on Windows
- Free or low-cost hosting for MVP
- PostgreSQL hosted locally for now, cloud DB for production
- No budget for paid APIs

Current status: Phase 1 + 2 (Auth + Student Profile + Admin) is fully built and running locally. The clickable HTML prototype (all 27 pages, 3 user roles) already exists

as a
design reference.

My skill level: Intermediate

Existing project structure:

- backend/ → FastAPI, SQLAlchemy, Alembic, 21 DB tables, 31 API endpoints
- frontend/ → React + Vite + Tailwind, 17 pages, Zustand state, Axios API layer
- Full SETUP.md and README.md already written

How I want you to work:

1. Do not build everything at once.
2. Work phase-by-phase.
3. Prefer the smallest reliable solution.
4. Separate MVP, V1, and future features.
5. Before writing code, explain the design and assumptions.
6. Ask only high-value clarification questions; if something is non-critical, make a clearly labeled assumption.
7. Do not invent libraries, APIs, credentials, files, or requirements.
8. When modifying code, preserve working functionality unless a change is intentional.
9. Give implementation steps in dependency order.
10. After each phase, give me a checkpoint and a definition of done.

Start by summarizing your understanding of the project in 5-10 bullets.
Then identify only the missing information required for the next task.
Do not write implementation code yet.

Phase 1 — Discovery & Problem Definition

Prompt 1A — Problem Statement

Act as a product strategist. Analyse this project:

CampusHire is a campus placement management platform for engineering colleges.
The system has three user roles: Student, Department Admin, and Super Admin.

Create:

1. Problem statement – what is broken today without this product?
2. Target users – who exactly are the three roles and what do they need?
3. User pain points – for each role, what are their top 3 frustrations with the current manual placement process?
4. Existing alternatives – what do colleges use today (spreadsheets, WhatsApp, other tools)?
5. Why this product should exist – what does it uniquely solve?
6. Core value proposition – one sentence per role
7. Key assumptions – what are we assuming that has not been validated?
8. Risks and unknowns – what could make this harder than expected?
9. What would make this project fail – top 3 reasons

10. 5 questions I should answer before development continues

Do not design features yet unless they directly support the problem.
Clearly distinguish facts, assumptions, and recommendations.

Prompt 1B — User Personas

Create 3 practical user personas for CampusHire.

Persona 1: The Student

Persona 2: The Department Admin (Placement Coordinator)

Persona 3: The Super Admin (Central Placement Cell Head)

For each persona include:

- Role / background
- Goals on this platform
- Main pain points with the current manual process
- Current workflow (what do they do today without this tool?)
- Technical ability (how comfortable are they with software?)
- What they need from CampusHire specifically
- What could make them reject or stop using the product

Do not create fictional details that are not useful for product decisions.
Keep it grounded in how Indian engineering college placement processes actually work.

Phase 2 — Requirements & PRD

Prompt 2A — Product Requirements Document

Create a lean Product Requirements Document (PRD) for CampusHire.

The system has three user roles:

- Student: Registers, verifies email via OTP, builds a 7-section profile (personal, academic, skills, projects, experience, certifications, preferences), views placement drives, fills application forms, tracks drive status, checks readiness score.
- Department Admin: Manages students in their department (add manually, bulk import via Excel), posts placement drives with eligibility criteria, builds Google-Forms-style application forms, views form responses, makes announcements, views reports.
- Super Admin: Manages departments, creates admin accounts, views institution-wide reports, audit log, and system settings.

Include:

1. Product overview
2. Problem
3. Goals
4. Non-goals (what this does NOT do)
5. Target users
6. Core user journeys (one per role)
7. Functional requirements (use IDs: FR-001, FR-002, etc.)
8. Non-functional requirements (performance, security, accessibility)
9. Business rules (e.g. only verified students can apply to drives)
10. Edge cases
11. Error states
12. Success metrics (how do we know CampusHire is working?)
13. Dependencies
14. Risks
15. Open questions

For every requirement use a unique ID.

Keep requirements testable and unambiguous.

Do not add features just because they sound useful.

Prompt 2B — Acceptance Criteria

Convert the following CampusHire requirements into testable acceptance criteria.

Requirements to convert:

- FR-001: Student can register with email, password, roll number, and department
- FR-002: Student must verify email via 6-digit OTP before account is activated
- FR-003: Student can complete their profile across 7 sections
- FR-004: Profile completion percentage is calculated and displayed
- FR-005: Department Admin can add a student manually
- FR-006: Department Admin can bulk import students via an Excel file
- FR-007: Super Admin can create a department with a unique department code
- FR-008: Super Admin can create a Department Admin account with a temporary password
- FR-009: All users can reset their password using a 3-step OTP flow

For each requirement provide:

- Requirement ID
- User story (As a [role], I want to [action] so that [benefit])
- Given (precondition)
- When (action)
- Then (expected outcome)
- Edge cases
- Validation notes

Keep criteria specific enough that a developer and tester would interpret them the same way.

Phase 3 — MVP Scope

Prompt 3A — Feature Classification

Act as a ruthless MVP product manager for CampusHire.

Project: CampusHire – campus placement management platform

Full feature list:

1. Student self-registration with OTP email verification
2. Student profile (7 sections: personal, academic, skills, projects, experience, certifications, preferences)
3. Profile completion percentage
4. Placement readiness score (based on profile + resume + assessment)
5. Resume builder with templates and PDF generation
6. AI resume analysis and scoring
7. Assessment engine (MCQs – aptitude, technical, communication, domain)
8. Placement drives (post, manage eligibility, rounds, deadlines)
9. Drive application flow (student applies, admin tracks status)
10. Google-Forms-style application form builder
11. Form responses viewer (summary, charts, individual, data table)
12. Excel bulk student import with validation preview
13. Announcements (target by dept/year)
14. In-app notification bell
15. Email notifications
16. Department management (Super Admin)
17. Admin account management (Super Admin)
18. Department-level reports and analytics
19. Institution-wide global reports (Super Admin)
20. Audit log
21. System settings (logo, email domain, email templates, resume templates)
22. Offer tracking (placement_offers table)
23. Dark mode
24. Mobile responsive layout

Classify every feature into:

- A. Must have for MVP (the product cannot demonstrate value without it)
- B. Useful but can wait for V1 (improves product, not blocking)
- C. Nice to have / future (good idea, not urgent)
- D. Remove unless evidence appears (questionable value for this use case)

For every decision explain the reasoning.

Then describe the smallest end-to-end MVP that delivers core value to all 3 roles.
The MVP must be demonstrable and usable, not merely a collection of screens.

Phase 4 — Architecture

Prompt 4A — Architecture Review

Review and validate the current architecture of CampusHire.

Current architecture:

- Frontend: React 18 + Vite 5 + Tailwind CSS 3 + Zustand + React Hook Form + Axios
- Backend: FastAPI 0.111 + Python 3.11 + SQLAlchemy 2.0 + Alembic
- Database: PostgreSQL 14+
- Auth: JWT (access token 30min + refresh token 7 days) + bcrypt passwords + OTP via email
- File storage: Local filesystem (uploads/ directory), S3-ready
- Email: SMTP (prints to terminal in dev mode when SMTP_USER is blank)

Current limitations:

- No rate limiting on any endpoint
- No background task queue (email is sent synchronously in request)
- No caching
- File uploads not yet implemented in the API
- No automated tests

Constraints:

- Solo developer
- Free/low-cost hosting for MVP
- Windows development environment
- Must stay maintainable as project grows to Phase 6

Review:

1. Is this architecture appropriate for the MVP scope?
2. What are the top 3 risks with the current architecture?
3. What should be changed now vs later?
4. Is there any unnecessary complexity that should be removed?
5. What is the simplest addition to make file uploads work?
6. What is the simplest addition to handle emails asynchronously?

For each recommendation:

- State whether it is required now or can wait
- Give the reason
- Give the implementation effort (small / medium / large)
- Name one reasonable alternative

Optimise for simplicity, maintainability, and ability to evolve to Phase 5 (placement drives).

Do not introduce microservices.

Prompt 4B — Database Model Review

Review the existing CampusHire database model for Phase 1 + 2.

Current tables (21 total):

Authentication: users, otp_verifications, password_reset_tokens, user_sessions

Institution: departments, admin_profiles, admin_department_mapping

Student profile: student_profiles, student_education, student_academic_records, skills, student_skills, student_projects, project_technologies, student_experiences, student_certifications, student_preferences, job_roles, student_preferred_roles, locations, student_preferred_locations

Key design decisions already made:

- users table is the central auth table for all 3 roles
- role is an ENUM: STUDENT, DEPT_ADMIN, SUPER_ADMIN
- registration_source records SELF_REGISTERED, ADMIN_ADDED, EXCEL_IMPORT
- must_change_password forces password change on first login for admin-created accounts
- import_job_id on student_profiles links bulk-imported students to their import job
- all IDs are UUIDs

Now design the Phase 3 additions needed for the Forms + Drives features:

- forms (application forms built by admins)
- form_sections and form_questions
- form_responses
- placement_drives
- drive_eligibility (which departments are eligible)
- drive_rounds
- drive_applications (student applies to drive)
- application_status_history

For each new table provide:

- Fields and types
- Primary keys
- Foreign keys
- Relationships to existing tables
- Unique constraints
- Required vs optional fields
- Indexes (which fields will be queried frequently?)
- Example records

Explain why each relationship exists.

Flag anything that is an assumption.

Prompt 4C — API Contract for Phase 3

Design the API contract for CampusHire Phase 3 – Forms and Placement Drives.

Existing API base: /api/v1

Existing routers: /auth, /student, /admin

New routers needed:

1. /admin/forms – form builder endpoints (admin only)
2. /admin/drives – placement drive endpoints (admin only)
3. /student/drives – student view of drives and applications

For each endpoint provide:

- Method (GET/POST/PATCH/DELETE)
- Path
- Purpose
- Authentication requirement (STUDENT / DEPT_ADMIN / SUPER_ADMIN / public)
- Request body or query parameters with types
- Response shape (key fields)
- Success HTTP status
- Common error statuses
- Key validation rules

Keep the API small. Do not create endpoints the MVP does not need.

Use consistent naming: snake_case for fields, plural nouns for collections.

Return paginated responses for lists (items, total, page, page_size, pages).

Phase 5 — Documentation

Prompt 5A — Documentation Structure Review

Review the existing documentation for CampusHire and identify gaps.

Existing documentation:

- README.md – setup guide, API reference, user flows, troubleshooting, phase roadmap
- CHANGELOG.md – all 20 build entries with file-level details
- SETUP.md – step-by-step setup guide for Windows (PostgreSQL, backend, frontend)
- OVERVIEW.md – prototype analysis (in the prototype folder)
- PROMPT_PLAYBOOK.md – this file

Missing documentation to create:

For each missing document:

1. What is its purpose?
2. What sections should it contain?
3. When should it be updated?
4. Who is the audience (developer, admin, user)?

Also identify:

- What is documented but outdated?
- What is missing that would block a new developer from contributing?
- What is missing that would block the Super Admin from operating the platform?

Prompt 5B — Architecture Decision Records

Write Architecture Decision Records (ADRs) for the following decisions already made in CampusHire. Use this format for each:

- Status: Accepted
- Context: [why this decision was needed]
- Decision: [what was decided]
- Alternatives considered: [what else was evaluated]
- Consequences: [what this means going forward – good and bad]
- Revisit conditions: [when should this decision be re-evaluated?]

Decisions to document:

ADR-001: Use FastAPI instead of Django

ADR-002: Use PostgreSQL with SQLAlchemy instead of MongoDB or a simpler ORM

ADR-003: Use JWT access + refresh tokens instead of session-based auth

ADR-004: Use Alembic for database migrations

ADR-005: Use React + Vite instead of Next.js

ADR-006: Use Tailwind CSS instead of a component library (MUI, Chakra, etc.)

ADR-007: Use Zustand instead of Redux for state management

ADR-008: Store uploaded files on the local filesystem for MVP (not S3)

ADR-009: Send emails synchronously in the request (not via a background queue) for MVP

ADR-010: Use UUID primary keys instead of integer auto-increment

Keep each ADR concise and useful to a future developer joining the project.

Phase 6 — UX / UI

Prompt 6A — Core User Flows

Act as a senior UX designer. Review the CampusHire user flows.

Product: CampusHire

Reference: A clickable HTML/CSS prototype already exists with 27 pages across 3 roles.

For each of the following flows, analyse and document:

1. Entry point
2. User goal
3. Steps (numbered)
4. System responses at each step
5. Loading states
6. Empty states
7. Error states
8. Success state
9. Exit / next action

Flows to document:

STUDENT FLOWS:

- S1: Student self-registration → OTP verification → first login → forced password change
- S2: Student completes their profile (all 7 tabs)

- S3: Student views placement drives and applies to one

ADMIN FLOWS:

- A1: Admin logs in → adds a department → creates a student account
- A2: Admin imports students via Excel (upload → validate → preview → confirm)
- A3: Admin builds an application form and links it to a drive

SUPER ADMIN FLOWS:

- SA1: Super Admin creates a department and assigns a department admin to it
- SA2: Super Admin resets a department admin's password

For each error state, specify: what caused it, what message the user sees, and what

action they can take next.

Do not start with visual styling. Start with usability and flow logic.

Prompt 6B — UI Component Audit

Review the existing CampusHire frontend UI as a senior UX/UI designer.

Existing components (frontend/src/components/):

Common: Button, Input, Select, Textarea, Card, MetricCard, Badge, Toast, ConfirmModal,

Skeleton, SkeletonCard, SkeletonTable, SkeletonMetricGrid

Layout: AppLayout, Sidebar, Topbar, ProtectedRoute

Existing pages (17 total):

Auth: Login, Register, OTPVerification, ResetPassword

Student: Dashboard, Profile (7-tab), Readiness, Notifications, Settings

Admin: Home, Students, AddStudent, ExcelUpload

Super Admin: Dashboard, Departments, AdminAccounts

Design system tokens (Tailwind):

- accent: #D85A30 (coral), teal: #0F6E56, amber: #854F0B, danger: #A32D2D, purple: #534AB7

- surface-0/1/2 (page/card backgrounds), border/border-strong, text-primary/secondary/muted

- Font: Inter

For each screen, review:

- Purpose clarity (does the user immediately understand what to do?)
- Components used (any missing? any duplicated?)
- Actions available (are all needed actions present? any unnecessary ones?)
- Responsive behaviour (does it work on mobile?)
- Loading state (does a skeleton show while data loads?)
- Empty state (what shows when there is no data?)
- Error state (what shows when an API call fails?)
- Accessibility (labels, focus, contrast, keyboard navigation)

Separate findings into: Critical / High / Medium / Low.
Do not recommend full redesigns. Recommend the smallest high-impact improvements.

Phase 7 — Frontend (Incremental)

Prompt 7A — Next Frontend Feature

I am working on CampusHire.

Current phase: Phase 3 – Forms & Placement Drives frontend

Goal for this task: Build the admin "My Forms" page and "Form Builder" page.

These pages already exist as HTML prototypes in CampusHire_Clickable_Prototype_2/:

- my-forms.html (Google-Forms-style list of forms)
- form-builder.html (question builder with 7 question types)

Existing relevant files:

- frontend/src/api/admin.ts (has adminApi – needs form endpoints added)
- frontend/src/components/common/ (Button, Input, Card, Badge, Toast, ConfirmModal, Skeleton)
- frontend/src/components/layout/ (AppLayout, Sidebar already has "My Forms" nav item)
- frontend/src/pages/admin/ (Home, Students, AddStudent, ExcelUpload already built)
- frontend/src/store/toastStore.ts (toast.success / toast.error helpers)

Constraints:

- Reuse existing components where possible
- Do not rewrite unrelated code
- Keep state management simple (local useState, no new Zustand store needed)
- Handle loading, empty, success, and error states
- Make it responsive (works on mobile)
- Match the existing design system (coral accent, Tailwind tokens)

Before implementation:

1. Inspect the context provided.
2. State your assumptions.
3. Propose the smallest implementation.
4. List the files you expect to create or change.

Then implement only this task.

After implementation:

- Explain what changed.
- Explain how to manually test it.
- List known limitations.
- Do not modify unrelated files.

Prompt 7B — Frontend Review

Review the current CampusHire frontend as a senior engineer.

Tech stack: React 18 + Vite + TypeScript + Tailwind CSS + Zustand + React Hook Form + Axios

Files to review are in: frontend/src/

Check:

- Component structure (are components appropriately sized and reusable?)
- Duplication (is any logic or UI duplicated across pages?)
- State management (is Zustand used only where needed? is local state used for simple cases?)
- Responsiveness (do all pages work on mobile? is the sidebar collapse working?)
- Accessibility (labels on inputs, focus management, button aria-labels, color contrast)
- Error handling (do all API calls have .catch() with user-visible feedback?)
- Loading/empty states (do all data-fetching pages show skeletons and empty states?)
- Performance (any unnecessary re-renders, large bundle imports, missing lazy loading?)
- Security-sensitive client behaviour (tokens stored safely? no sensitive data in URL params?)
- Maintainability (would a new developer understand the code structure quickly?)

Separate findings into: Critical / High / Medium / Low.

Do not refactor everything. Recommend the smallest high-impact fixes first.

Phase 8 — Backend (Incremental)

Prompt 8A — Next Backend Feature

I am working on CampusHire.

Current phase: Phase 3 – Forms & Placement Drives backend

Goal for this task: Build the backend for the Forms feature.

Forms allow Department Admins to create Google-Forms-style application forms linked to placement drives. Students fill these forms when applying. Admins view responses.

A form has:

- title, description, status (draft/published), linked_drive_id
- sections (ordered groups of questions)
- questions per section (7 types: short_answer, paragraph, multiple_choice, checkboxes, dropdown, date, file_upload)
- responses (one per student per form, contains answers keyed by question ID)

Existing backend files:

- backend/app/models/ (user.py, auth.py, department.py, student.py – all working)
- backend/app/services/admin_service.py (add form CRUD here)
- backend/app/api/v1/endpoints/admin.py (add form endpoints here)
- backend/app/schemas/admin.py (add form schemas here)
- backend/alembic/ (run alembic revision to add new tables)

Before implementation:

1. List the new models needed (tables, fields, relationships).
2. List the new Pydantic schemas needed (request + response).
3. List the new service functions needed.
4. List the new API endpoints needed with method + path + purpose.
5. List which existing files change and what changes.

Then implement only the foundation: models + schemas + one complete endpoint (e.g. POST /admin/forms to create a form).

After implementation:

- Show how to test it manually using the Swagger UI at <http://localhost:8000/api/docs>
- List known limitations
- State the next recommended step

Prompt 8B — Vertical Slice

Build one complete vertical slice for CampusHire.

Feature: Student applies to a placement drive

A vertical slice means this feature works end-to-end:

Frontend (student clicks "Apply now") →

API request (POST /api/v1/student/drives/{drive_id}/apply) →

Backend validation (is student eligible? has student already applied? is deadline passed?) →

Business logic (create drive_application record, set status to APPLIED) →

Database write →

API response (application confirmed) →

Frontend state update (button changes to "Applied", status badge updates)

Before coding, list:

- All files that will be created or changed (frontend + backend)
- The complete data flow from click to database write to screen update
- All validation rules the backend must enforce
- All error cases that must be handled

After coding, provide:

- Files changed
- How to run and test it manually (step by step)

- Known limitations
- Next vertical slice recommendation

Phase 9 — Integration & End-to-End MVP

Prompt 9A — Integration Check

I need to verify the integration between the CampusHire frontend and backend.

Existing frontend: React + Vite on `http://localhost:5173`

Existing backend: FastAPI on `http://localhost:8000`

Vite proxy config: `/api` → `http://localhost:8000`, `/uploads` → `http://localhost:8000`

Integration points to verify:

1. Authentication flow:

- `POST /api/v1/auth/login` returns { `access_token`, `refresh_token`, `role`, `user_id` }
- Frontend stores tokens in `localStorage`, attaches Bearer header via Axios interceptor
- 401 response triggers token refresh via `POST /api/v1/auth/refresh`
- Role-based redirect after login (`STUDENT`→`/dashboard`, `DEPT_ADMIN`→`/admin`, `SUPER_ADMIN`→`/super-admin`)

2. Student profile flow:

- `GET /api/v1/student/profile` returns `FullProfileOut` (all 7 sections)
- `PATCH /api/v1/student/profile` updates personal info
- PUT endpoints for each of the 7 profile sections

3. Admin student management:

- `GET /api/v1/admin/students?search=&department_id=&year=&page=&page_size=`
- `POST /api/v1/admin/students` (add single student)
- `POST /api/v1/admin/students/import/preview` (multipart file upload)
- `POST /api/v1/admin/students/import/confirm` (multipart file upload)

4. Super Admin flows:

- `GET/POST/PATCH/DELETE /api/v1/admin/departments`
- `GET/POST/PATCH /api/v1/admin/accounts`
- `POST /api/v1/admin/accounts/{id}/reset-password`

For each integration point:

- Check that the request/response shapes match between frontend api/ files and backend schemas/
- Check that error responses are handled in the frontend (toast shown, not silent failure)
- Check that loading states are shown during API calls
- Check that CORS is not blocking any request

Return:

1. Working correctly

2. Mismatches found (list each one with the fix)
3. Missing error handling (list each one)
4. End-to-end manual test checklist (step-by-step for a new tester)

Prompt 9B — MVP Readiness Review

Act as a skeptical product and engineering reviewer.

Review whether CampusHire Phase 1 + 2 is truly a working MVP.

What has been built:

- Backend: FastAPI + PostgreSQL, 21 tables, 31 endpoints, JWT auth, OTP, Excel import
- Frontend: React + Vite, 17 pages (4 auth, 5 student, 4 admin, 3 super-admin)
- All pages: loading states, empty states, error handling via toast notifications
- SETUP.md, README.md with full instructions

Check:

1. Does the core user journey work end-to-end?
 - Student registers → OTP verified → logs in → completes profile → views readiness
 - Admin logs in → adds students → views student list
 - Super Admin logs in → creates department → creates admin account
2. Are must-have requirements complete for all 3 roles?
3. Are major failure paths handled? (wrong password, expired OTP, duplicate email, etc.)
4. Is data persisted correctly to PostgreSQL?
5. Is the UX understandable without training?
6. Are there obvious security risks? (exposed secrets, missing auth checks, SQL injection)
7. Can a new developer follow SETUP.md and run the project?
8. Is deployment to a cloud environment possible without major changes?

Return:

1. Ready / Not ready
2. Blocking issues (must fix before calling this an MVP)
3. Non-blocking issues (should fix but not urgent)
4. Exact next actions in priority order
5. Recommended next phase to build

Do not recommend polishing before blocking issues are fixed.

Phase 10 — Testing, Debugging & Security

Prompt 10A — Test Strategy

Create a testing strategy for CampusHire Phase 1 + 2.

Current state: No automated tests exist. Manual testing only.

Tech stack:

- Backend: FastAPI + Python (pytest available)
- Frontend: React + Vite + TypeScript (Vitest available)

Prioritise tests for the MVP. Include:

1. Unit tests – which business logic functions need unit tests first?
(e.g. profile completion calculator, OTP hash verification, Excel row validator)
2. Integration tests – which API endpoints need integration tests?
(focus on auth flow, student profile save, Excel import)
3. API tests – which endpoint behaviours must be verified?
(auth, role guards, validation errors, edge cases)
4. UI/component tests – which React components need tests?
(focus on forms, error states, protected routes)
5. End-to-end tests – what is the minimum E2E test that proves the MVP works?
6. Manual exploratory tests – what should be tested manually before each release?
(list a checklist for a non-technical tester)

For each category:

- What should be tested
- What does NOT need testing yet (save effort)
- Recommended tool (pytest, httpx, Vitest, Playwright)
- Estimated effort (small / medium / large)

Focus testing effort on business-critical paths and failure-prone logic.

Prompt 10B — Debug Template

Help me debug this CampusHire issue.

Expected behavior:

[DESCRIBE WHAT SHOULD HAPPEN]

Actual behavior:

[DESCRIBE WHAT IS ACTUALLY HAPPENING]

Error / log output:

[PASTE THE FULL ERROR MESSAGE OR STACK TRACE HERE]

Relevant files / code:
[PASTE THE RELEVANT CODE SECTION OR FILE PATH]

Environment:

- OS: Windows
- Python: 3.11
- Node: 18+
- Backend running on: http://localhost:8000
- Frontend running on: http://localhost:5173
- Database: PostgreSQL (local)

Do not guess the root cause immediately.

First:

1. List the likely causes (ordered by probability).
2. Rank them.
3. Tell me what evidence would confirm or refute each.
4. Identify the smallest diagnostic step I can take right now.
5. Then propose the fix.

Do not change unrelated code.

Prompt 10C — Security Review

Perform a security review of CampusHire MVP.

Tech stack: FastAPI backend, React frontend, PostgreSQL, JWT auth, bcrypt passwords.

Known security measures already in place:

- Passwords hashed with bcrypt (passlib)
- OTPs stored as SHA-256 hashes (not plain text)
- JWT access tokens expire in 30 minutes
- Refresh tokens stored as hashes in DB, revoked on logout
- Role-based access control on all endpoints (require_student, require_dept_admin, require_super_admin dependencies)
- CORS configured with explicit allowed origins
- Pydantic validation on all request bodies

Check for:

- Authentication / authorization flaws (can a student access admin endpoints?)
- SQL injection (are all queries using parameterised SQLAlchemy ORM calls?)
- XSS (is any user-provided content rendered as raw HTML in React?)
- Sensitive data exposure (are tokens/passwords ever logged or returned in responses?)
- Secrets in code (is the SECRET_KEY or DB password ever hardcoded?)
- Unsafe file uploads (when file upload is implemented, what validation is needed?)
- Weak validation (what fields have no length/format limits?)
- Insecure API behaviour (are there any endpoints that should require auth but don't?)

- Excessive permissions (can a dept admin access another department's students?)
- Rate limiting (is there any protection against brute-force login attempts?)
- Logging of sensitive data (are OTPs or passwords ever printed to logs?)
- Production configuration risks (what must change before deploying publicly?)

Classify each finding by severity: Critical / High / Medium / Low

Give a practical, specific remediation for each finding.

Do not claim the system is secure – identify what was and was not reviewed.

Phase 11 — Deployment

Prompt 11A — Deployment Plan

Create a low-cost deployment plan for CampusHire MVP.

Constraints:

- Solo developer / small team
- Prefer free or low-cost services for MVP
- Must support PostgreSQL
- Python backend (FastAPI with uvicorn)
- React frontend (static build output)
- No Kubernetes or Docker required for MVP
- Windows local development, Linux production is fine

Current local setup:

- Backend: `uvicorn app.main:app --port 8000`
- Frontend: `npm run dev (Vite)` – built output is `frontend/dist/`
- Database: PostgreSQL local
- File uploads: stored in `backend/uploads/`

Cover:

- Frontend hosting (where to host the React static build?)
- Backend hosting (where to host the FastAPI app?)
- Database (where to host PostgreSQL?)
- Environment variables (how to manage secrets in production?)
- Domain (what is the simplest way to get a domain and HTTPS?)
- HTTPS (how to get SSL certificates?)
- Build commands (what commands run before deployment?)
- Migrations (how to run alembic upgrade head in production?)
- File/upload storage (how to handle uploaded files in production?)
- Logging (where do server logs go?)
- Backups (how to back up the PostgreSQL database?)
- Monitoring (how to know if the server is down?)
- Rollback strategy (how to revert a bad deployment?)
- Cost estimate (what is the expected monthly cost?)

For each item:

1. Recommend one specific service or approach
2. Give the reason
3. Give one reasonable alternative

4. State the cost (free / approx. per month)

Prefer the simplest architecture that can handle the expected MVP traffic (a few hundred students per college).

Prompt 11B — Production Launch Checklist

Create a production launch checklist for CampusHire.

The application is:

- FastAPI backend with 31 endpoints, JWT auth, PostgreSQL, Alembic migrations
- React frontend with 17 pages, Zustand state, Axios API layer
- File uploads to local filesystem
- OTP email via SMTP

Include these categories:

Pre-launch (code and config):

- [] All environment variables set in production (not using .env defaults)
- [] SECRET_KEY is a strong random value (not the dev placeholder)
- [] DEBUG=False in production
- [] DATABASE_URL points to production PostgreSQL
- [] SMTP credentials configured (real email sending enabled)
- [] ALLOWED_ORIGINS updated to production frontend URL
- [] .env file not committed to git

Database:

- [] alembic upgrade head run on production database
- [] First Super Admin account created (via SQL or seeder)
- [] At least one department created
- [] Database backup configured

Security:

- [] HTTPS enabled (not HTTP)
- [] CORS restricted to production domain only
- [] No sensitive data in logs
- [] Rate limiting considered for /auth/login
- [] File upload directory not publicly browsable

Frontend:

- [] npm run build completes without errors
- [] VITE_API_BASE_URL points to production backend
- [] No console.log statements with sensitive data

Testing:

- [] Manual smoke test: register → OTP → login → profile → admin → super-admin
- [] Password reset flow tested
- [] Excel import tested with real file

Documentation:

- [] README.md updated with production URLs
- [] CHANGELOG.md updated with release version

Rollback:

- [] Previous working build retained
- [] Database backup taken before migration

Mark each item as: Required for MVP launch / Can wait for V1

Phase 12 — Post-MVP Iteration

Prompt 12A — V2 Backlog Prioritisation

CampusHire Phase 1 + 2 (Auth + Student Profile + Admin) is live and working.

Current user feedback (hypothetical based on the domain):

- Students want to know which drives they are eligible for based on their CGPA
- Admins want to post placement drives and have students apply through the platform
- Admins want to build application forms and see responses
- Super Admin wants to see placement statistics across all departments
- Students want email notifications when a new drive is posted

Known issues:

- No placement drive feature exists yet (Phase 5 in roadmap)
- No application form builder exists yet (Phase 3 in roadmap)
- No assessment/readiness scoring engine exists yet (Phase 4 in roadmap)
- Readiness score is a placeholder (calculated only from profile completion)
- Email notifications are not implemented (SMTP configured but no trigger system)
- No file upload for profile photos or resumes
- No resume builder

Planned phases not yet built:

- Phase 3: Forms + Placement Drives
- Phase 4: Assessment engine + real readiness score
- Phase 5: Application tracking + offer management
- Phase 6: Announcements + notifications + reports + audit log

Create a prioritised V2 backlog.

Score each phase/feature on:

- User impact (1-5)
- Frequency of use (1-5)
- Business value to the institution (1-5)
- Engineering effort (1=easy, 5=hard)
- Risk reduction (does this fix a current gap?)

Do not prioritise features merely because they were requested.

Recommend what should actually be built next and why.

Output a prioritised table with reasoning.

Prompt 12B — Safe Refactor

Review the CampusHire backend services layer for technical debt.

Files to review:

- backend/app/services/auth_service.py
- backend/app/services/student_service.py
- backend/app/services/admin_service.py

Identify:

- Real problems that will cause bugs or security issues as the project grows
- Symptoms that do not need fixing now
- High-risk code (e.g. code that changes many records at once without transactions)
- Duplication (is any logic repeated across service files?)
- Performance issues (any N+1 query patterns? any missing DB indexes?)
- Maintainability issues (is any function too long or doing too many things?)

For each recommended refactor:

- Why now? (why not later?)
- Expected benefit
- Risk of making the change
- Estimated effort (small / medium / large)
- Safe implementation order (what must be done first?)

Do not recommend a large rewrite unless there is a strong measurable reason.
Focus on the highest-impact, lowest-risk improvements.

The Best Prompt Pattern for Any CampusHire Coding Task

Use this template whenever asking AI to make a specific code change.

I am working on CampusHire.

Current phase: [e.g. Phase 3 – Forms feature]

Goal for this task: [ONE SPECIFIC GOAL – e.g. "Add the form builder API endpoint"]

Existing behavior: [What currently exists or what currently happens]

Relevant files:

- backend/app/services/admin_service.py
- backend/app/schemas/admin.py
- backend/app/api/v1/endpoints/admin.py
- [add others as needed]

Constraints:

- Do not change any existing working endpoints
- Match the existing code style (SQLAlchemy ORM, Pydantic v2 schemas, service layer pattern)
- Use the existing toast and error handling patterns on the frontend

```
Before implementation:
1. Inspect the context provided.
2. State your assumptions.
3. Propose the smallest implementation that achieves the goal.
4. List the files you expect to create or change.

Then implement only this task.

After implementation:
- Explain what changed and why.
- Explain how to test it manually (step by step).
- List known limitations.
- Do not modify unrelated functionality.
```

Prompts to Avoid

These prompt patterns produce poor results for CampusHire. Avoid them:

Weak prompt	Why it fails	Use instead
"Build the complete Phase 3 feature."	Too broad, no scope	Break into: models → schemas → services → endpoints → frontend, one at a time
"Make the app production ready."	Undefined criteria	Use the Production Launch Checklist prompt above
"Fix everything in the frontend."	No prioritisation	Use the Frontend Review prompt, then fix Critical issues only
"Add AI resume analysis."	No success metric	Define: what model, what input, what output, how is it measured
"Make it more professional."	Subjective	Specify: which page, which element, what does professional mean here
"Rewrite the backend to be cleaner."	High risk, undefined benefit	Use the Safe Refactor prompt, pick one function at a time
"Do whatever you think is best."	Gives away product decisions	Always specify constraints and acceptance criteria

AI Conversation Management

For CampusHire, use separate conversations per phase to avoid context noise:

Conversation	Topic
1	Product discovery + PRD
2	Architecture + database + API

Conversation	Topic
3	UX/UI design
4	Frontend implementation
5	Backend implementation
6	Testing + security
7	Deployment
8	Post-MVP V2 planning

Context Handoff Prompt

Use this when starting a new conversation about CampusHire:

This is a continuation of an existing project.

Project: CampusHire – campus placement management platform

Current phase: [STATE THE CURRENT PHASE]

Project summary:

- 3 user roles: Student, Department Admin, Super Admin
- Backend: FastAPI + PostgreSQL, 21 tables, 31 endpoints, JWT auth + OTP
- Frontend: React + Vite + Tailwind, 17 pages, Zustand + Axios
- Phase 1 + 2 (Auth + Student Profile) is complete and running locally

Decisions already made:

- FastAPI over Django (speed, auto-docs, async-ready)
- PostgreSQL with SQLAlchemy 2.0 ORM + Alembic migrations
- JWT access (30min) + refresh tokens (7 days)
- UUID primary keys throughout
- Tailwind CSS with custom design tokens (coral accent #D85A30)
- Zustand for global state (auth + toast), local useState for everything else
- Replace all alert()/confirm() with custom toast/modal components

Current architecture:

backend/

```

app/main.py → FastAPI app, CORS, router
app/api/v1/router.py → includes auth, student, admin routers
app/core/ → config, security, email, deps
app/models/ → user, auth, department, student (21 tables)
app/schemas/ → auth, student, admin, common
app/services/ → auth_service, student_service, admin_service

```

frontend/

```

src/App.tsx → all routes by role
src/api/ → client.ts, auth.ts, student.ts, admin.ts
src/store/ → authStore.ts, toastStore.ts
src/components/common/ → Button, Input, Card, Badge, Toast, ConfirmModal,
Skeleton

```



```
src/components/layout/ → AppLayout, Sidebar, Topbar, ProtectedRoute  
src/pages/ → auth/, student/, admin/, super-admin/
```

Current files/features: [DESCRIBE WHAT YOU JUST BUILT OR WHAT IS CURRENTLY BROKEN]

Do not redesign decisions without a reason.

First summarize your understanding in 5-10 bullets.

Then identify only the missing information required for the next task.

Final Master Checklist

Use this before calling any phase "done":

Phase 1 + 2 (Auth + Profile) — Current Phase

- ☒ Problem is clearly defined
- ☒ Target users are known (Student, Dept Admin, Super Admin)
- ☒ MVP scope is intentionally small (Phase 1+2 only)
- ☒ Requirements are documented
- ☒ Architecture is documented (README.md)
- ☒ Database model is documented (CHANGELOG.md)
- ☒ API contract is defined (README.md)
- ☒ Frontend foundation is stable
- ☒ Backend foundation is stable
- ☒ At least one vertical slice works end-to-end (register → OTP → login → profile)
- ☒ Core MVP journeys work for all 3 roles
- ☒ Loading/empty/error states exist on all pages
- ☐ Important business logic is tested (no automated tests yet)
- ☐ Security basics are reviewed (not formally reviewed yet)
- ☒ Environment variables/secrets are handled safely (.env not committed)
- ☒ Deployment steps are documented (SETUP.md, README.md)
- ☐ Rollback/backup approach is known
- ☒ README lets another developer run the project

Phase 3 (Forms + Drives) — Next Phase

- ☐ Form builder API designed
- ☐ Form builder frontend built
- ☐ Placement drive API designed
- ☐ Placement drive frontend built
- ☐ Student drive view built
- ☐ Application flow works end-to-end

One rule to remember: Don't use AI as a one-shot code generator. Use it as an engineering partner operating inside a controlled process.

The strongest workflow is: Understand → Define → Scope → Design → Document → Build small → Test → Review → Integrate → Deploy → Learn → Improve

Files to Attach With Each Prompt

This section tells you exactly which project files to attach alongside each prompt when running it in Kiro or any AI tool. Attach files using the `#File` context key or the attachment button in the chat input.

Phase 0 — Master Context Prompt

Attach: Nothing — paste the prompt as-is. All context is written inside it.

Phase 1 — Discovery

Prompt	Files to attach
1A — Problem Statement	Nothing — context is inside the prompt
1B — User Personas	Nothing — context is inside the prompt

Phase 2 — Requirements & PRD

Prompt	Files to attach
2A — PRD	Nothing — all 3 role descriptions are inside the prompt
2B — Acceptance Criteria	Nothing — the 9 requirements are listed inside the prompt

Phase 3 — MVP Scope

Prompt	Files to attach
3A — Feature Classification	Nothing — all 24 features are listed inside the prompt

Phase 4 — Architecture

Prompt	Files to attach
4A — Architecture Review	<code>backend/app/main.py</code> · <code>backend/app/api/v1/router.py</code> · <code>backend/requirements.txt</code>
4B — Database Model Review	<code>backend/app/models/user.py</code> · <code>backend/app/models/student.py</code> · <code>backend/app/models/department.py</code> · <code>backend/app/models/auth.py</code>
4C — API Contract for Phase 3	<code>backend/app/api/v1/endpoints/admin.py</code> · <code>backend/app/schemas/admin.py</code>

Phase 5 — Documentation

Prompt	Files to attach
5A — Documentation Gap Review	<code>README.md</code> · <code>CHANGELOG.md</code> · <code>SETUP.md</code>
5B — Architecture Decision Records	<code>backend/requirements.txt</code> · <code>frontend/package.json</code> · <code>backend/app/core/config.py</code>

Phase 6 — UX / UI

Prompt	Files to attach
6A — Core User Flows	<code>frontend/src/App.tsx</code>
6B — UI Component Audit	<code>frontend/src/components/common/Button.tsx</code> · <code>frontend/src/components/layout/Sidebar.tsx</code> · <code>frontend/src/pages/student/Profile.tsx</code> · <code>frontend/src/pages/admin/Students.tsx</code>

Phase 7 — Frontend

Prompt	Files to attach
7A — Next Frontend Feature (Forms)	<code>frontend/src/api/admin.ts</code> · <code>frontend/src/components/common/Button.tsx</code> · <code>frontend/src/pages/admin/Students.tsx</code> (<i>as a pattern reference</i>)
7B — Frontend Review	<code>frontend/src/App.tsx</code> · <code>frontend/src/store/authStore.ts</code> · <code>frontend/src/pages/student/Profile.tsx</code> · <code>frontend/src/pages/admin/Students.tsx</code>

Phase 8 — Backend

Prompt	Files to attach
8A — Next Backend Feature (Forms)	<code>backend/app/services/admin_service.py</code> · <code>backend/app/schemas/admin.py</code> · <code>backend/app/api/v1/endpoints/admin.py</code> · <code>backend/app/models/student.py</code>
8B — Vertical Slice (Student applies to drive)	<code>backend/app/models/student.py</code> · <code>backend/app/services/admin_service.py</code> · <code>frontend/src/api/admin.ts</code> · <code>frontend/src/pages/student/Dashboard.tsx</code>

Phase 9 — Integration

Prompt	Files to attach
9A — Integration Check	<code>frontend/src/api/client.ts</code> · <code>frontend/src/api/auth.ts</code> · <code>frontend/src/api/admin.ts</code> · <code>backend/app/core/deps.py</code>
9B — MVP Readiness Review	<code>README.md</code> · <code>frontend/src/App.tsx</code> · <code>backend/app/main.py</code>

Phase 10 — Testing, Debugging & Security

Prompt	Files to attach
10A — Test Strategy	<code>backend/app/services/auth_service.py</code> · <code>backend/app/services/student_service.py</code>
10B — Debug Template	The specific file where the bug is + paste the full error in the prompt
10C — Security Review	<code>backend/app/core/security.py</code> · <code>backend/app/core/deps.py</code> · <code>backend/app/api/v1/endpoints/auth.py</code> · <code>frontend/src/api/client.ts</code> · <code>frontend/src/store/authStore.ts</code>

Phase 11 — Deployment

Prompt	Files to attach
11A — Deployment Plan	<code>README.md</code> · <code>backend/requirements.txt</code> · <code>frontend/package.json</code>
11B — Production Launch Checklist	<code>backend/.env.example</code> · <code>backend/app/main.py</code>

Phase 12 — Post-MVP

Prompt	Files to attach
12A — V2 Backlog Prioritisation	<code>README.md</code> (the Phase Roadmap section is enough)
12B — Safe Refactor	The specific service file you want reviewed: <code>backend/app/services/auth_service.py</code> or <code>backend/app/services/admin_service.py</code>

Coding Task Template (General)

Prompt	Files to attach
Any backend change	The specific <code>models/</code> , <code>services/</code> , <code>schemas/</code> , <code>endpoints/</code> files being changed

Prompt	Files to attach
Any frontend change	The specific <code>api/</code> , <code>pages/</code> , <code>components/</code> files being changed
Any bug fix	The broken file + paste the full error message inside the prompt

General Rule

Prompt type	What to attach
Planning / strategy prompts	Nothing — context is written inside the prompt
Architecture prompts	<code>main.py</code> · <code>models/</code> files · <code>requirements.txt</code>
Backend feature prompts	The relevant <code>models/</code> · <code>services/</code> · <code>schemas/</code> · <code>endpoints/</code> files
Frontend feature prompts	The relevant <code>api/</code> · <code>pages/</code> · <code>components/</code> files
Review / audit prompts	The files being reviewed
Debug prompts	The broken file + the full error message pasted in the prompt
Integration prompts	Both frontend <code>api/</code> files AND backend <code>endpoints/</code> files